

Unfolding Computational Graphs

- A Computational Graph is a way to formalize the structure of a set of computations
 - Such as mapping inputs and parameters to outputs and loss
- We can unfold a recursive or recurrent computation into a computational graph that has a repetitive structure
 - Corresponding to a chain of events
- Unfolding this graph results in sharing of parameters across a deep network structure

Example of unfolding a recurrent equation

- Classical form of a dynamical system is

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}; \boldsymbol{\theta})$$

- where $\mathbf{s}^{(t)}$ is called the state of the system
- Equation is recurrent because the definition of $\mathbf{s}$ at time t refers back to the same definition at time $t-1$
- For a finite no. of time steps τ , the graph can be unfolded by applying the definition $\tau-1$ times
 - E.g, for $\tau = 3$ time steps we get

$$\begin{aligned}\mathbf{s}^{(3)} &= f(\mathbf{s}^{(2)}; \boldsymbol{\theta}) \\ &= f(f(\mathbf{s}^{(1)}; \boldsymbol{\theta}); \boldsymbol{\theta})\end{aligned}$$

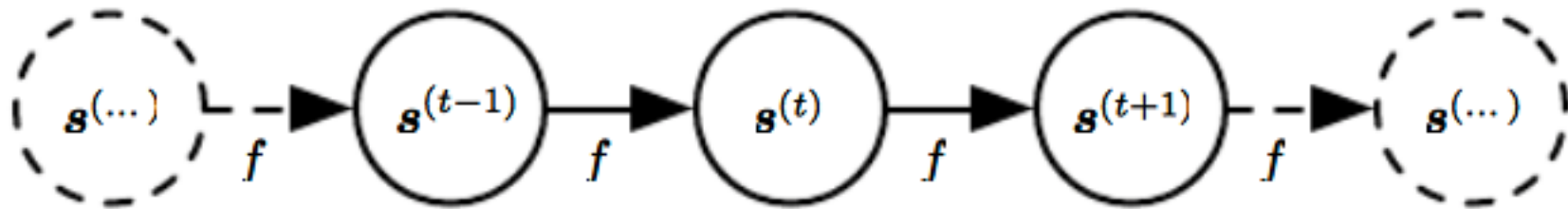
- Unfolding equation by repeatedly applying the definition in this way has yielded expression without recurrence
 - $\mathbf{s}^{(1)}$ is ground state and $\mathbf{s}^{(2)}$ computed by applying f
- Such an expression can be represented by a traditional acyclic computational graph (as shown next)

Unfolded dynamical system

- The classical dynamical system described by

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}; \boldsymbol{\theta}) \text{ and } \mathbf{s}^{(3)} = f(f(\mathbf{s}^{(1)}; \boldsymbol{\theta}); \boldsymbol{\theta})$$

is illustrated as an unfolded computational graph



- Each node represents state at some time t
- Function f maps state at time t to the state at $t+1$
- The same parameters (the same value of $\boldsymbol{\theta}$ used to parameterize f) are used for all time steps

Dynamical system driven by external signal

- As another example, consider a dynamical system driven by external (input) signal $\mathbf{x}^{(t)}$

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}, \mathbf{x}^{(t)}; \boldsymbol{\theta})$$

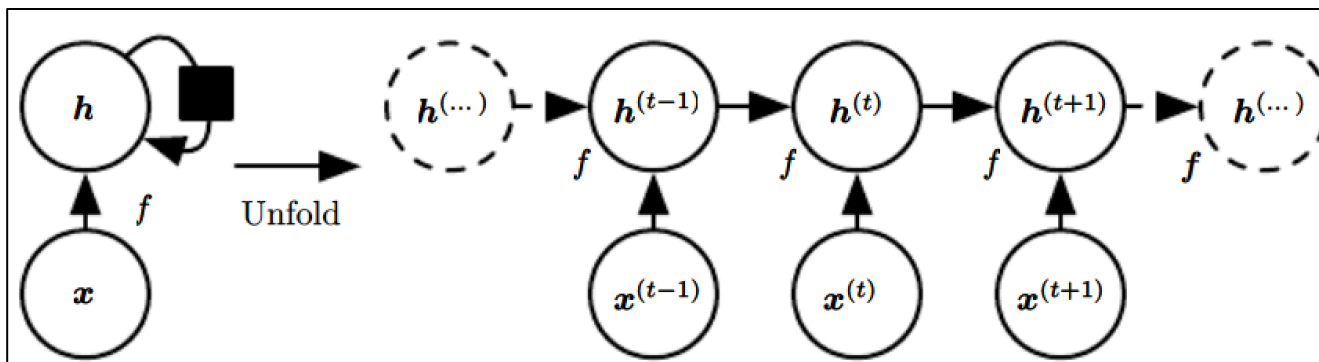
- State now contains information about the whole past input sequence
 - Note that the previous dynamic system was simply $\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}; \boldsymbol{\theta})$
- Recurrent neural networks can be built in many ways
 - Much as almost any function is a feedforward neural network, any function involving recurrence can be considered to be a recurrent neural network

Defining values of hidden units in RNNs

- Many recurrent neural nets use same equation (as dynamical system with external input) to define values of hidden units
 - To indicate that the state is hidden rewrite using variable h for state:

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}; \theta)$$

- Illustrated below:



- Typical RNNs have extra architectural features such as output layers that read information out of state h to make predictions

A recurrent network with no outputs

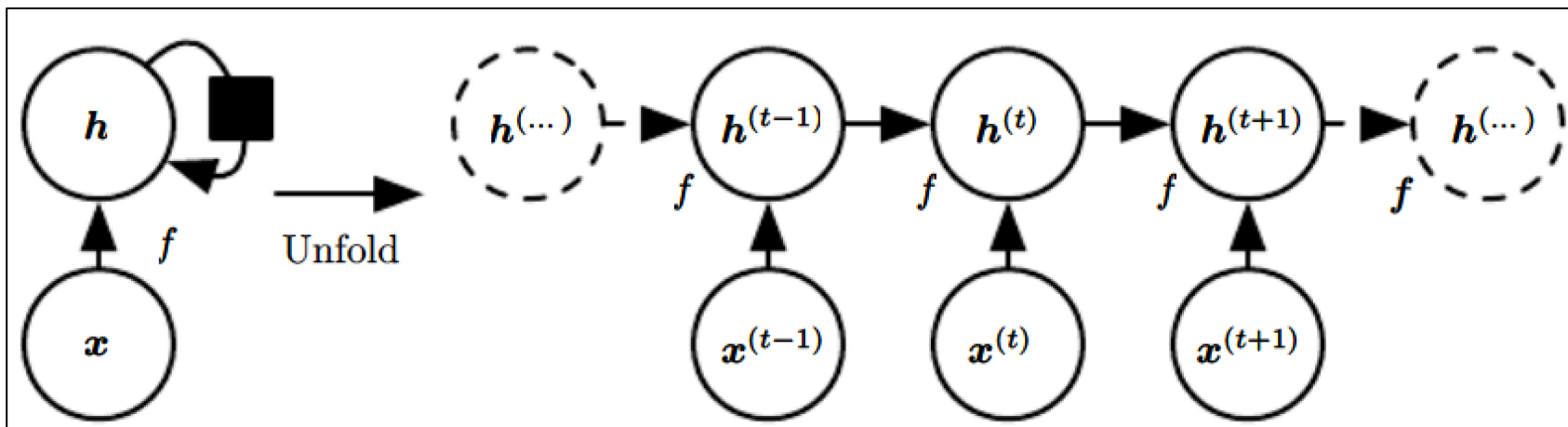
- This network just processes information from input x by incorporating it into state h that is passed forward through time

Circuit diagram:

Black square indicates
Delay of one time step

Unfolded computational graph:

each node is now
associated with one time instance



Typical RNNs will add extra architectural features such as output layers to read information out of the state h to make predictions

Predicting the Future from the Past

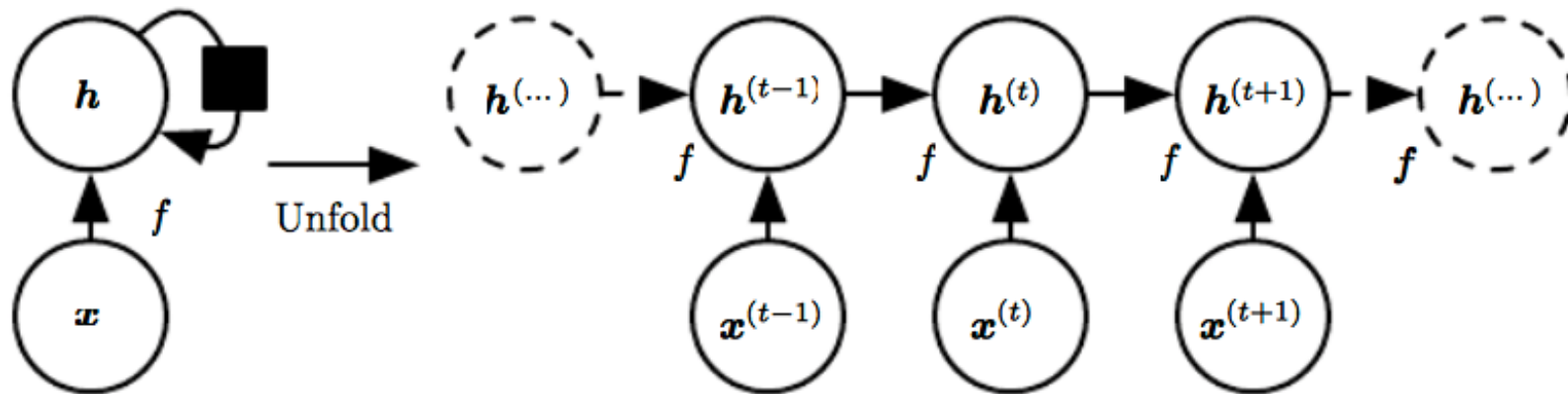
- When RNN is required to perform a task of predicting the future from the past, network typically learns to use $\mathbf{h}^{(t)}$ as a lossy summary of the task-relevant aspects of the past sequence of inputs upto t
- The summary is in general lossy since it maps a sequence of arbitrary length $(\mathbf{x}^{(t)}, \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(2)}, \mathbf{x}^{(1)})$ to a fixed length vector $\mathbf{h}^{(t)}$

Information Contained in Summary

- Depending on criterion, summary keeps some aspects of past sequence more precisely than other aspects
- Examples:
 - RNN used in statistical language modeling, typically to predict next word from past words
 - it may not be necessary to store all information upto time t but only enough information to predict rest of sentence
 - Most demanding situation: we ask $\mathbf{h}^{(t)}$ to be rich enough to allow one to approximately recover the input sequence as in autoencoders

Unfolding: from circuit diagram to computational graph

- Equation $\mathbf{h}^{(t)} = f(\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}; \boldsymbol{\theta})$ can be written in two different ways: circuit diagram or an unfolded computational graph



- Unfolding is the operation that maps a circuit to a computational graph with repeated pieces
- The unfolded graph has a size dependent on the sequence length

Process of Unfolding

- We can represent unfolded recurrence after t steps with a function $g^{(t)}$:

$$\begin{aligned} \mathbf{h}^{(t)} &= g^{(t)}(\mathbf{x}^{(t)}, \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(2)}, \mathbf{x}^{(1)}) \\ &= f(\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}; \boldsymbol{\theta}) \end{aligned}$$

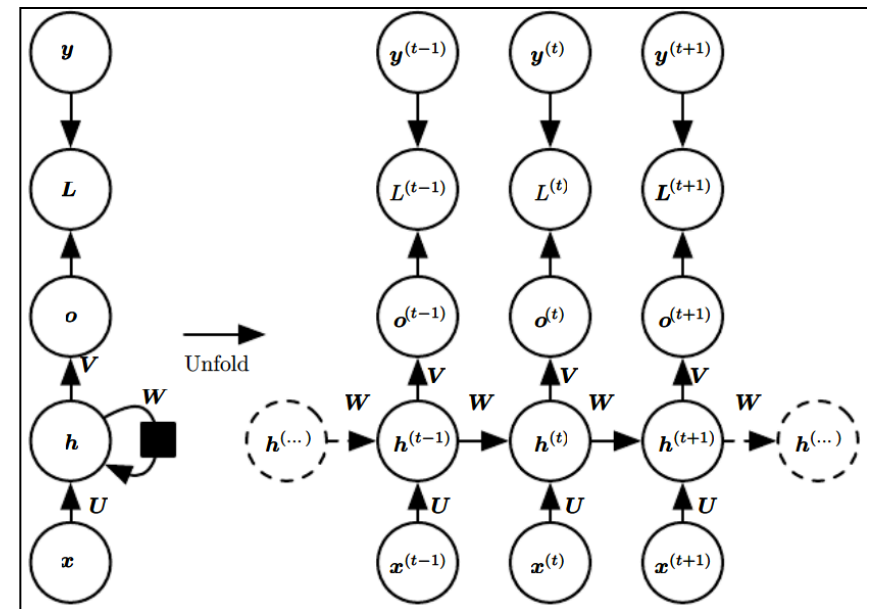
- Function $g^{(t)}$ takes in whole past sequence $(\mathbf{x}^{(t)}, \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(2)}, \mathbf{x}^{(1)})$ as input and produces the current state but the unfolded recurrent structure allows us to factorize $g^{(t)}$ into repeated application of a function f
- The unfolding process introduces two major advantages as discussed next.

Unfolding process allows learning a single model

- The unfolding process introduces two major advantages:
 1. Regardless of sequence length, learned model has same input size
 - because it is specified in terms of transition from one state to another state rather than specified in terms of a variable length history of states
 2. Possible to use same function f with same parameters at every step
- These two factors make it possible to learn a single model f
 - that operates on all time steps and all sequence lengths
 - rather than needing separate model $g^{(t)}$ for all possible time steps
- Learning a single shared model allows:
 - Generalization to sequence lengths that did not appear in the training
 - Allows model to be estimated with far fewer training examples than would be required without parameter sharing

Unfolding Computational Graphs

- A Computational Graph is a way to formalize the structure of a set of computations
 - Such as mapping inputs and parameters to outputs and loss
- We can unfold a recursive or recurrent computation into a computational graph that has a repetitive structure
 - Corresponding to a chain of events
- Unfolding this graph results in sharing of parameters across a deep network structure



$$\begin{aligned}
 \mathbf{o}^{(t)} &= \mathbf{c} + V\mathbf{h}^{(t)} \\
 \mathbf{h}^{(t)} &= \tanh(\mathbf{a}^{(t)}) \\
 \mathbf{a}^{(t)} &= \mathbf{b} + W\mathbf{h}^{(t-1)} + U\mathbf{x}^{(t)}
 \end{aligned}$$